МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1

Выполнил:
Студент группы ИБМ3-34Б

Фролова М.П.

Преподаватель:
Правдина А.Д.

Бурмистрова М.В.

Формулировка задания:

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A, B, C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.
4. Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.
5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.
6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования.

Листинг программы:

Файл lab1.py (Python):

```
import sys
import math

def get_coef(name, index):

    if len(sys.argv) > index + 1:
        try:
            return float(sys.argv[index + 1])
        except ValueError:
            print(f"Некорректное значение коэффициента {name} в командной строке.
Введите его вручную.")

    while True:
        s = input(f"Введите коэффициент {name}: ").strip().replace(',', '.')
        try:
            return float(s)
        except ValueError:
            print(f"Ошибка: '{s}' не число. Повторите ввод.")


def get_roots(A, B, C):
```

```
    roots = []

    if A == 0:
        if B == 0:
            if C == 0:
                return "infinite"
            return []
        t = -C / B
        if t > 0:
            roots += ([math.sqrt(t), -math.sqrt(t)])
        elif t == 0:
            roots.append(0.0)
        return roots

    D = B * B - 4 * A * C

    if D < 0:
        return roots

    if D == 0:
        t = -B / (2 * A)
        if t > 0:
            roots += ([math.sqrt(t), -math.sqrt(t)])
        elif t == 0:
            roots.append(0.0)
        return roots

    sqrtD = math.sqrt(D)
    t1 = (-B + sqrtD) / (2 * A)
    t2 = (-B - sqrtD) / (2 * A)

    for t in (t1, t2):
        if t > 0:
            r = math.sqrt(t)
            roots += ([r, -r])
        elif t == 0:
            roots.append(0.0)

    return roots

def format_equation(A, B, C):

    parts = []

    if A != 0:
        if A == 1:
            parts.append("x⁴")
        elif A == -1:
            parts.append("-x⁴")
        else:
            parts.append(f"{A:g}x⁴")
```

```
    if B != 0:
        if B < 0:
            sign = "-"
        else:
            sign = "+"

        b = abs(B)
        if not parts:
            if b == 1:
                if B > 0:
                    parts.append("x²" )
                else:
                    parts.append("-x²")
            else:
                if B < 0:
                    parts.append(f"{'-'}{b:g}x²")
                else:
                    parts.append(f"{''}{b:g}x²")
        else:
            if b == 1:
                parts.append(f" {sign} x²")
            else:
                parts.append(f" {sign} {b:g}x²")

    if C != 0 or not parts:
        if C < 0:
            sign = "-"
        else:
            sign = "+"
        c = abs(C)
        if not parts:
            if C < 0:
                parts.append(f"{'-'}{c:g}")
            else:
                parts.append(f"{''}{c:g}")
        else:
            parts.append(f" {sign} {c:g}")

    return " ".join(parts) + " = 0"

def main():
    print("РЕШЕНИЕ БИКВАДРАТНОГО УРАВНЕНИЯ")

    A = get_coef("A", 0)
    B = get_coef("B", 1)
    C = get_coef("C", 2)

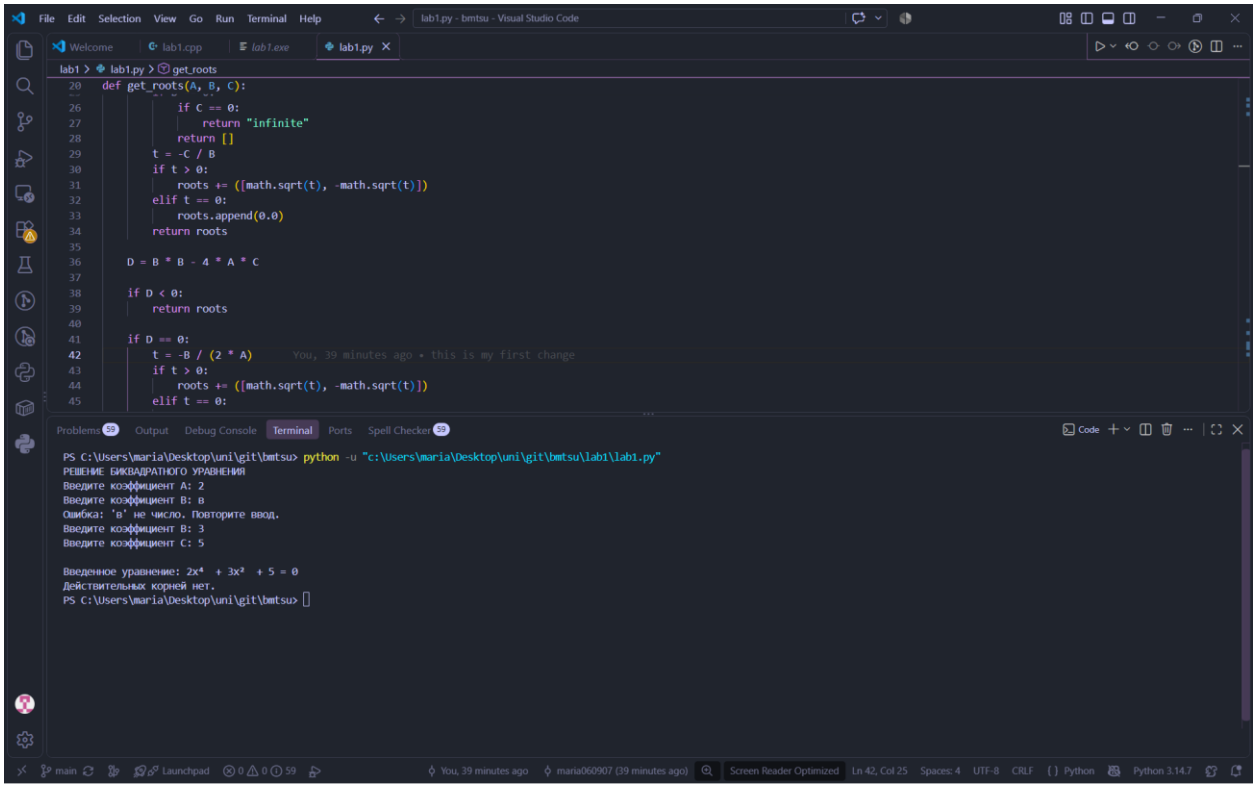
    print(f"\nВведенное уравнение: {format_equation(A, B, C)}")

    result = get_roots(A, B, C)
```

```
    if result == "infinite":
        print("Уравнение имеет бесконечно много решений (x может быть любым).")
    elif not result:
        print("Действительных корней нет.")
    else:
        true_roots = sorted(set(round(r, 10) for r in result))
        print("Действительные корни есть:")
        for r in true_roots:
            print(f"  x = {r}")

if __name__ == "__main__":
    main()
```

Вывод программы:



Файл lab1.cpp (C++):

```
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <algorithm>
#include <set>
#include <windows.h>
```

```cpp
bool isNumber(const std::string& s) {
    if (s.empty()) {
        return false;
    }

    int dotCount = 0;
    int start = 0;

    if (s[0] == '-') {
        if (s.length() == 1) {
            return false;
        }
        start = 1;
    }

    for (size_t i = start; i < s.length(); ++i) {
        if (s[i] == '.' || s[i] == ',') {
            dotCount++;
            if (dotCount > 1) {
                return false;
            }
        }
        else if (!std::isdigit(s[i])) {
            return false;
        }
    }

    return true;
}

class BiquadraticSolver {
private:
    double A, B, C;

public:
    BiquadraticSolver(double a, double b, double c) : A(a), B(b), C(c) {}

    bool infiniteRoots() const {
        return (A == 0 && B == 0 && C == 0);
    }

    std::vector<double> solve() const {
        std::vector<double> roots;

        if (infiniteRoots()) {
            return roots;
        }

        if (A == 0) {
            if (B != 0) {
                double t = -C / B;
```

```
            addRoots(t, roots);
        }
        return roots;
    }

    double D = B * B - 4 * A * C;
    if (D < 0) {
        return roots;
    }

    if (D == 0) {
        double t = -B / (2 * A);
        addRoots(t, roots);
    } else {
        double sqrtD = std::sqrt(D);
        double t1 = (-B + sqrtD) / (2 * A);
        double t2 = (-B - sqrtD) / (2 * A);

        addRoots(t1, roots);
        addRoots(t2, roots);
    }

    std::sort(roots.begin(), roots.end());

    std::vector<double> uniqueRoots;
    for (double r : roots) {
        if (uniqueRoots.empty() || std::abs(r - uniqueRoots.back()) > 1e-9) {
            if (std::abs(r) < 1e-9){
                r = 0.0;
            }
            uniqueRoots.push_back(r);
        }
    }
    return uniqueRoots;
}

void formatEquation() const {
    std::cout << "Введенное уравнение: ";
    bool first = true;

    if (A != 0) {
        if (A == 1) {
            std::cout << "x⁴";
        }
        else if (A == -1) {
            std::cout << "-x⁴";
        }
        else {
            std::cout << A << "x⁴";
        }
        first = false;
```

```
        }

        if (B != 0) {
            if (!first) {
                if (B > 0) {
                    std::cout << " + ";
                }
                else {
                    std::cout << " - ";
                }
                double b_abs = std::abs(B);
                if (b_abs == 1) {
                    std::cout << "x²";
                }
                else {
                    std::cout << b_abs << "x²";
                }
            } else {
                if (B == 1) {
                    std::cout << "x²";
                }
                else if (B == -1) {
                    std::cout << "-x²";
                }
                else {
                    std::cout << B << "x²";
                }
            }
            first = false;
        }

        if (C != 0 || first) {
            if (!first) {
                if (C > 0) {
                    std::cout << " + ";
                }
                else {
                    std::cout << " - ";
                }
                std::cout << std::abs(C);
            } else {
                std::cout << C;
            }
        }

        std::cout << " = 0\n";
    }

private:
    void addRoots(double t, std::vector<double>& roots) const {
        if (t > 0) {
```

```
            double r = std::sqrt(t);
            roots.push_back(r);
            roots.push_back(-r);
        } else if (t == 0) {
            roots.push_back(0.0);
        }
    }
};

static double getCoef(const std::string& name) {
        std::string input;
        while (true) {
            std::cout << "Введите коэффициент " << name << ": ";
            std::cin >> input;


            if (isNumber(input)) {
                std::replace(input.begin(), input.end(), ',', '.');
                return std::stod(input);
            }

            std::cout << "Ошибка: '" << input << "' не число. Повторите ввод.\n";
        }
    }

int main(int argc, char* argv[]) {

    SetConsoleCP(65001);
    SetConsoleOutputCP(65001);


    std::cout << "РЕШЕНИЕ БИКВАДРАТНОГО УРАВНЕНИЯ\n\n";

    double A = getCoef("A");
    double B = getCoef("B");
    double C = getCoef("C");

    BiquadraticSolver solver(A, B, C);

    std::cout << "\n";
    solver.formatEquation();

    if (solver.infiniteRoots()) {
        std::cout << "Уравнение имеет бесконечно много решений (x может быть
любым).\n";
    } else {
        std::vector<double> roots = solver.solve();
        if (roots.empty()) {
            std::cout << "Действительных корней нет.\n";
        } else {
            std::cout << "Действительные корни есть:\n";
```

```
            for (double r : roots) {
                std::cout << "   x = " << r << "\n";
            }
        }
    }

    return 0;
}
```

Вывод программы:

